Advanced C: Part 1 of 3

INDEX Next

Comprehensive listing of interrupts and hardware details.

CONTENTS OF PART ONE

ROM BIOS CALLS

The ROM BIOS (Basic Input Output System) provides device control for the PC's major devices (disk, video, keyboard, serial port, printer), allowing a programmer to communicate with these devices without needing detailed knowledge of their operation. The ROM routines are accessed via the Intel 8088/86 software generated interrupts. The interrupts 10H through to 1AH each access a different routine.

Parameters are passed to and from the BIOS routines using the 8088/86 CPU registers. The routines normally preserve all registers except AX and the flags. Some registers are altered if they return values to the calling process.

```
ROM BIOS INTERRUPT ROUTINES
10      Video routines
11      Equipment Check
12      Memory Size Determination
13      Diskette routines
14      Communications routines
15      Cassette
16      Keyboard routines
17      Printer
18      Cassette BASIC
19      Bootstrap loader
1A      Time of Day
```

The interrupts which handle devices are like a gateway which provide access to more than one routine. The routine executed will depend upon the contents of a particular CPU register. Each of the software interrupt calls use the 8088/86 register contents to determine the desired function call. It is necessary to use a C definition of the CPU programming model, this allows the registers to be initialised with the correct values before the interrupt is generated. The definition also provides a convienent place to store the returned register values. Luckily, the definition has already been created, and resides in the header file dos.h. It is a union of type REGS, which has two parts, each structures.

One structure contains the eight bit registers (accessed by .h.), whilst the other structure contains the

16 bit registers (accessed by .x.) To generate the desired interrupt, a special function call has been provided. This function accepts the interrupt number, and pointers to the programming model union for the entry and return register values. The following program demonstrates the use of these concepts to set the display mode to 40x25 color.

```
#include <dos.h>
union REGS regs;
main()
{
        regs.h.ah = 0;
        regs.h.al = 1;
        int86( 0x10, &regs, &regs );
        printf("Fourty by Twenty-Five color mode.");
}
```

VIDEO ROM BIOS CALLS

The ROM BIOS supports many routines for accessing the video display. The following table illustrates the use of software interrupt 0x10.

SCREEN DISPLAY MODE FUNCTION CALL (regs.h.ah = 0)

regs.h.al	Screen Mode
0	40.25 BW
1	40.25 CO
2	80.25 BW
3	80.25 CO
4	320.200 CO
5	320.200 BW
6	640.200 BW
7	Mono-chrome
8	160.200 16col PCjr
9	320.200 16col PCjr
A	640.200 4col PCjr
D	320.200 16col EGA
E	640.200 16col EGA
F	640.350 mono EGA
10	640.350 16col EGA

Note: The change screen mode function call also has the effect of clearing the video screen! The other routines associated with the video software interrupt 0x10 are, (Bold represents return values)

Function Call	regs.h.ah	Entry/Exit Values
Set display mode	0	Video mode in al
Set cursor type	1	Start line=ch, end line=cl (Block cursor, cx = 020C)
Set cursor position	2	Row,column in dh,dl, page number
Read cursor position	3	Page number in bh, dh,dl on exit
Read light pen pos	4	ah=0 light pen not active ah=1 light pen activated dh,dl has row,column ch has raster line (0-199) bx has pixel column (0-139,639)
Select active page	5	New page number in al
Scroll active page up	6	Lines to scroll in al(0=all) upper left row,column in ch,cl lower right row,col in dh,dl

```
                                        attribute for blank line in bh
Scroll active page dn   7               Same as for scroll up
Read char and attr      8               Active page in bh
                                        al has character
                                        ah has attribute
Write char and attr     9               Active page in bh
                                        number of characters in cx
                                        character in al
                                        attribute in bl
Write character         0a              Active page in bh
                                        number of characters in cx
                                        character in al
Set color palette       0b              Palette color set in bh
                                        (graphics) color value in bl
Write dot               0c              Row,column in dx,cx
                                        color of pixel in al
Read dot                0d              Row,column in dx,cx color in al
Write teletype          0e              Character in al, active page in b
                                        foregrnd color(graphics) in bl
Return video state      0f              Current video mode in al
                                        columns in ah,active page in bh
```

PORT ACCESS

The C language can be used to transfer data to and from the contents of the various registers and controllers associated with the IBM-PC. These registers and control devices are port mapped, and are accessed using special **IN** and **OUT** instructions. Most C language support library's include functions to do this. The following is a brief description of how this may be done.

```
/* #include <conio.h> */
outp( Port_Address, value); /* turboC uses outportb() */
value = inp( Port_address); /* and inportb() */
```

The various devices, and their port values, are shown below,

Port Range	**Device**
00 - 0f	DMA Chip 8737
20 - 21	8259 PIC
40 - 43	Timer Chip 8253
60 - 63	PPI 8255 (cassette, sound)
80 - 83	DMA Page registers
200 - 20f	Game I/O Adapter
278 - 27f	Reserved
2f8 - 2ff	COM2
378 - 37f	Parallel Printer
3b0 - 3bf	Monochrome Display
3d0 - 3df	Color Display
3f0 - 3f7	Diskette
3f8 - 3ff	COM1

PROGRAMMING THE 6845 VIDEO CONTROLLER CHIP

The various registers of the 6845 video controller chip, resident on the CGA card, are

Port Value	Register Description
3d0	6845 registers
3d1	6845 registers
3d8	DO Register (Mode control)
3d9	DO Register (Color Select)
3da	DI Register (Status)
3db	Clear light pen latch
3dc	Preset light pen latch

PROGRAMMING EXAMPLE FOR THE BORDER COLOR

The register which controls the border color is the Color Select Register located at port 3d9. Bits 0 - 2 determine the border color. The available colors and values to use are,

Value	Color	Value	Color
0	Black	8	Dark Grey
1	Blue	9	Light Blue
2	Green	0a	Light Green
3	Cyan	0b	Light Cyan
4	Red	0c	Light Red
5	Magenta	0d	Light Magenta
6	Brown	0e	Yellow
7	Light Grey	0f	White

The following program will set the border color to blue.

```
#include <conio.h> /* needed for outp() */
#include <stdio.h> /* needed for getchar() */
#include <dos.h> /* for REGS definition */
#define CSReg 0x3d9
#define BLUE 1
void cls()
{
        union REGS regs;
        regs.h.ah = 15; int86( 0x10, &regs, &regs );
        regs.h.ah = 0; int86( 0x10, &regs, &regs );
}
main()
{
        cls();
        printf("Press any key to set border color to blue.\n");
        getchar();
        outp( CSReg, BLUE );
}
```

PROGRAMMING EXAMPLE FOR 40x25 COLOR MODE

The 6845 registers may be programmed to select the appropiate video mode. This may be done via a ROM BIOS call or directly. The values for each of the registers to program 40.25 color mode are,

```
38,28,2d,0a,1f,6,19,1c,2,7,6,7,0,0,0,0
```

The default settings for the registers for the various screen modes can be found in ROM BIOS listing's and technical reference manuals.

To program the various registers, first write to the address register at port 3d4, telling it which register you are programming, then write the register value to port 3d5. The mode select register must also be programmed. This is located at port 3d8. The sequence of events is,

1: Disable the video output signal

2: Program each register

3: Enable video output, setting mode register

The following program illustrates how to do this,

```
#include <conio.h> /* needed for outp() */
#include <stdio.h> /* needed for getchar() */
#include <process.h> /* needed for system calls */
#define MODE_REG 0x3d8
#define VID_DISABLE 0
#define FOURTY_25 0X28
#define ADDRESS_REG 0x3d4
#define REGISTER_PORT 0x3d5
static int mode40x25[] = {0x38,0x28,0x2d,0x0a,0x1f,6,0x19,0x1c,
2,7,6,7,0,0,0,0 };

void cls()
{
        union REGS regs;
        regs.h.ah = 15; int86( 0x10, &regs, &regs );
        regs.h.ah = 0; int86( 0x10, &regs, &regs );
}

main()
{
        int loop_count = 0;
        cls();
        printf("Press a key to set 40x25 color mode.\n");
        getchar();
        outp(MODE_REG,VID_DISABLE); /*disable video signal */
        for( ; loop_count < 0x0e; ++loop_count) {
                outp( ADDRESS_REG, loop_count ); /* set up CRT register *
                outp( REGISTER_PORT, mode40x25[loop_count]); /* write to
        }
        outp( MODE_REG, FOURTY_25); /* switch in mode now */;
        printf("Press key to exit.\n"); getchar();
}
```

The program should update the video_mode byte stored at 0040:0049 to indicate the change in video state. This type of low level programming is an example of code required for embedded applications (ie, ROM CODE running without DOS or the ROM BIOS chip present).

USING A ROM BIOS INTERRUPT CALL TO SET THE VIDEO MODE

The video chip may be also manipulated by using the ROM BIOS calls. A BIOS interrupt call follows the following syntax,

```
int86( interrupt_number, &regs1, &regs2);
where int86() is the function call,
interrupt_number is the interrupt to generate
&regs1 defines the input register values
&regs2 defines the returned register values
```

Int 10h is the interrupt to use for video calls. We will be using the set mode routine, so the values are,

```
regs.h.ah = 0; /* set mode function call */
regs.h.al = 1; /* set mode to 40x25 color */
```

The following program illustrates how this all fits together,

```
#include <dos.h>
#include <stdio.h>
#include <process.h>
union REGS regs;
void cls()
{
        regs.h.ah = 15; int86( 0x10, &regs, &regs );
        regs.h.ah = 0; int86( 0x10, &regs, &regs );
}

main()
{
        cls();
        printf("Setting mode to 40x25 color.\n");
        printf("Press any key....\n");
        getchar();
        regs.h.ah = 0;
        regs.h.al = 1;
        int86( 0x10, &regs, &regs);
        printf("Mode has been set.\n");
}
```

Calls to the ROM BIOS Int 10h Set Video Mode routine also update the video_mode flag stored at 0040:0049.

EXTENDED VIDEO BIOS CALLS

The following video calls are not supported on all machines. Some calls are only present if an adapter card is installed (ie, EGA or VGA card).

```
AH = 10h Select Colors in EGA/VGA
        AL = 1 BL = color register (0 - 15), BH = color to set
        AL = 2 ES:DX ptr to change all 16 colors and overscan number
        AL = 3 BL = color intensity bit. 0 = intensity, 1 = blinking
        (VGA systems only)
        AL = 7 BL = color register to get into BH
        AL = 8 BH = returned overscan value
        AL = 9 ES:DX = address to store all 16 colors + overscan number
        AL = 10h BX = color register to set; ch/cl/dl = green/blue/red
        AL = 12h ES:DX = pointer to change color registers,
                BX = 1st register to set,
```

```
                CX = number of registers involved
        AL = 13h BL = 0, set color page mode in BH
                BL = 1, set page number specified by BH
        AL = 15h BX = color register to read; ch/cl/dl = grn/blue/red
        AL = 17h ES:DX = pointer where to load color registers
                BX = 1st register to set,
                CX = number of registers involved
        AL = 1Ah get color page information; BL = mode, BH = page number
AH = 11h Reset Mode with New Character Set
        AL = 0 Load new character set; ES:BP pointer to new table
                BL/BH = how many blocks/bytes per character
                CX/DX = number of characters/where to start in block
        AL = 1 BL = block to load the monochrome character set
        AL = 2 BL = block to load the double width character set
        AL = 3 BL = block to select related to attribute
        AL = 4 BL = block to load the 8x16 set (VGA)
        AL = 10h - 14h Same as above, but must be called after a set mode
        AL = 20h ES:BP pointer to a character table, using int 1Fh pointe
                BL = 0, DL = number of rows, 1=14rows, 2=25rows, 3=43rows
                CX = number of bytes per character in table
        AL = 22h use 8x14 character set, BL = rows
        AL = 23h use double width character set, BL = rows
        AL = 24h use 8x16 character set, BL = rows
        Get table pointers and other information
        AL = 30h ES:BP = returned pointer; CX = bytes/character; DL = row
        BH = 0, get int 1Fh pointer, BH = 1, get int 43h pointer
        BH = 2, get 8x14 BH = 3, get double width
        BH = 4, get double width BH = 5, get mono 9x14
        BH = 6, get 8x16 (VGA) BH = 7, get 9x16 (VGA)
AH = 12h Miscellaneous functions, BL specifies sub-function number
        BL = 10h Get info, BH = 0, now color mode, 1 = now monochrome
        CH/CL = information bits/switches
        BL = 20h Set print screen to work with EGA/VGA
        Functions for VGA only (BL = 30-34h, AL returns 12h)
        BL = 30h Set number of scan lines, 0=200, 1=350, 2=400
        BL = 31h AX = 0/1 Allow/Prevent palette load with new mode
        BL = 32h AL = 0/1 Video OFF/ON
        BL = 33h AL = 0/1 Grey scale summing OFF/ON
        BL = 34h AL = 0/1 Scale cursor size to font size OFF/ON
        BL = 35h Switch between adapter and motherboard video
        AL = 0, adapter OFF, ES:DX = save state area
        AL = 1, motherboard on
        AL = 2, active video off, ES:DX = save area
        AL = 3, inactive video on, ES:DX = save area
        BL = 36h, AL = 0/1 Screen OFF/ON
AH = 13h Write Character string(cr,lf,bell and bs as operators)
        AL = 0/1 cursor NOT/IS moved, BL = attribute for all chars
        AL = 2/3 cursor NOT/IS moved, string has char/attr/char/attr
                BH = page number, 0 = 1st page, CX = number of characters
                DH/DL = row/column to start, ES:BP = pointer to string
AH = 14h LCD Support
        AL = 0h ES:DI = pointer to font table to load
                BL/BH = which blocks/bytes per character
                CX/DX = number of characters/where to start in block
        AL = 1h BL = block number of ROM font to load
        AL = 2h BL = enable high intensity
AH = 15h Return LCD information table Pointer in ES:DI, AX has screen mod
AH = 16h GET/SET display type (VGA ONLY)
        AL = 0h Get display type BX = displays used, AL = 1Ah
        AL = 1h Set display type BX = displays to use, returns AL = 1Ah
AH = 1Bh Get Video System Information (VGA ONLY)
        Call with BX = 0; ES:DI ptr to buffer area
AH = 1Ch Video System Save & Restore Functions (VGA ONLY)
        AL = 0 Get buffer size
```

```
                AL = 1 Save system, buffer at ES:BX
                AL = 2 Restore system, buffer at ES:BX
                        CX = 1 For hardware registers
                        CX = 2 For software states
                        CX = 4 For colors and DAC registers
```

SYSTEM VARIABLES IN LOW MEMORY

```
0040:0000       Address of RS232 card COM1
0040:0002       Address of RS232 card COM2
0040:0004       Address of RS232 card COM3
0040:0006       Address of RS232 card COM4
0040:0008       Address of Printer port LPT1
0040:000A       Address of Printer port LPT2
0040:000C       Address of Printer port LPT3
0040:0010       Equipment bits: Bits 13,14,15 = number of printers
                        12 = game port attached
                        9,10,11 = number of rs232 cards
                        6,7 = number of disk drives
                        4,5 = Initial video mode
                                (00=EGA, 01=CGA40,10=CGA80, 11=MONO)
                        3,2 = System RAM size
                        1 = Maths co-processor
                        0 = Boot from drives?
0040:0013       Main RAM Size
0040:0015       Channel IO size
0040:0017       Keyboard flag bits (byte) 7=ins, 6=caps, 5=num, 4=scrll,
                        3=ALT, 2=CTRL, 1=LSHFT, 0=RSHFT (toggle states)
0040:0018       Keyboard flag bits (byte) (depressed states)
0040:0019       Keyboard ALT-Numeric pad number buffer area
0040:001A       Pointer to head of keyboard queue
0040:001C       Pointer to tail of keyboard queue
0040:001E       15 key queue (head=tail, queue empty)
0040:003E       Recalibrate floppy drive, 1=drive0, 2=drv1, 4=drv2, 8=drv3
0040:003F       Disk motor on status, 1=drive0, 2=drv1, 4=drv2, 8=drv3
                        80h = disk write in progress
0040:0040       Disk motor timer 0=turn off motor
0040:0041       Disk controller return code
                        1=bad cmd, 2=no address mark, 3=cant write, 4=sector not
                        8=DMA overrun,9=DMA over 64k
                        10h=CRC error,20h=controller fail, 40h=seek fail,80h=time
0040:0042       Disk status bytes (seven)
0040:0049       Current Video Mode (byte)
0040:004A       Number of video columns
0040:004C       Video buffer size in bytes
0040:004E       Segment address of current video memory
0040:0050       Video cursor position page 0, bits8-15=row,bits0-7=column
0040:0052       Video cursor position page 1, bits8-15=row,bits0-7=column
0040:0054       Video cursor position page 2, bits8-15=row,bits0-7=column
0040:0056       Video cursor position page 3, bits8-15=row,bits0-7=column
0040:0058       Video cursor position page 4, bits8-15=row,bits0-7=column
0040:005A       Video cursor position page 5, bits8-15=row,bits0-7=column
0040:005C       Video cursor position page 6, bits8-15=row,bits0-7=column
0040:005E       Video cursor position page 7, bits8-15=row,bits0-7=column
0040:0060       Cursor mode, bits 8-12=start line, 0-4=end line
0040:0062       Current video page number
0040:0063       Video controller base I/O port address
0040:0065       Hardware mode register bits
0040:0066       Color set in CGA mode
0040:0067       ROM initialisation pointer
```

```
0040:0069        ROM I/O segment address
0040:006B        Unused interrupt occurrences
0040:006C        Timer low count (every 55milliseconds)
0040:006E        Timer high count
0040:0070        Timer rollover (byte)
0040:0071        Key-break, bit 7=1 if break key is pressed
0040:0072        Warm boot flag, set to 1234h for warm boot
0040:0074        Hard disk status byte
0040:0075        Number of hard disk drives
0040:0076        Head control byte for hard drives
0040:0077        Hard disk control port (byte)
0040:0078 - 7B   Countdown timers for printer timeouts LPT1 - LPT4
0040:007C - 7F   Countdown timers for RS232 timeouts, COM1 - COM4
0040:0080        Pointer to beginning of keyboard queue
0040:0082        Pointer to end of keyboard queue
Advanced Video Data, EGA/VGA
0040:0084        Number of rows - 1
0040:0085        Number of pixels per character * 8
0040:0087        Display adapter options (bit3=0 if EGA card is active)
0040:0088        Switch settings from adapter card
0040:0089 - 8A   Reserved
0040:008B        Last data rate for diskette
0040:008C        Hard disk status byte
0040:008D        Hard disk error byte
0040:008E        Set for hard disk interrupt flag
0040:008F        Hard disk options byte, bit0=1 when using a single
                 controller for both hard disk and floppy
0040:0090        Media state for drive 0 Bits 6,7=data transfer rate
                        (00=500k,01=300k,10=250k)
                        5=two steps?(80tk as 40k) 4=media type 3=unused
                        2,1,0=media/drive state (000=360k in 360k drive)
                        (001=360k in 1.2m drive) (010=1.2m in 1.2m drive)
                        (011=360k in 360k drive) (100=360k in 1.2m drive)
                        (101=1.2m in 1.2m drive) (111=undefined )
0040:0091        Media state for drive 1
0040:0092        Start state for drive 0
0040:0093        Start state for drive 1
0040:0094        Track number for drive 0
0040:0095        Track number for drive 1
0040:0096 - 97   Advanced keyboard data
0040:0098 - A7   Real time clock and LAN data
0040:00A8 - FF   Advanced Video data
0050:0000        Print screen status 00=ready,01=in progress,FFh=error
```

LIBRARIES

A library is a collection of useful routines or modules which perform various functions. They are grouped together as a single unit for ease of use. If the programmer wishes to use a module contained in a library, they only need to specify the module name, observing the correct call/return conditions. C programmers use librarys all the time, they are just unaware of it. The vast majority of routines such as printf, scanf, etc, are located in a C library that the linker joins to the code generated by the compiler. In this case, we will generate a small library which contains the modules *rdot()* and *wdot()*.

These modules read and write a dot to the video screen respectively. The programmer could retain the object code for these seperately, instead of placing them in a library, but the use of a library makes life simpler in that a library contains as many routines as you incorporate into it, thus simplifying the linking process. In other words, a library just groups object modules together under a

common name.

The following programs describe the source code for the modules *rdot()* and *wdot()*.

```
#include <dos.h>
union REGS regs;
wdot( int row, int column, unsigned int color )
{
        regs.x.dx = row; regs.x.cx = column;
        regs.h.al = color; regs.h.ah = 12;
        int86( 0x10, &regs, &regs);
}

unsigned int rdot( int row, int column )
{
        regs.x.dx = row; regs.x.cx = column;
        regs.h.ah = 13; int86( 0x10, &regs, &regs);
        return( regs.h.al );
}
```

The modules are compiled into object code. If we called the source code VIDCALLS.C then the object code will be VIDCALLS.OBJ. To create a library requires the use of the LIB.EXE program. This is invoked from the command line by typing **LIB**

Enter in the name of the library you wish to create, in this case **VIDCALLS**. The program will check to see if it already exists, which it doesn't, so answer **Y** to the request to create it. The program requests that you now enter in the name of the object modules. The various operations to be performed are,

```
to add an object module +module_name
to remove an object module -module_name
```

Enter in **+vidcalls**. It is not necessary to include the extension. The program then requests the list file generated by the compiler when the original source was compiled. If a list file was not generated, just press enter, otherwise specify the list file. That completes the generation of the VIDCALLS.LIB library.

The routines in VIDCALLS.LIB are incorporated into user programs as follows,

```
/* source code for program illustrating use of VIDCALLS.LIB */
#include <dos.h>
union REGS regs;
extern void wdot();
extern int rdot();
main()
{
        int color;
        regs.h.ah = 0; /* set up 320 x 200 color */
        regs.h.al = 4;
        int86( 0x10, &regs, &regs );
        wdot( 20, 20, 2 );
        color = rdot( 20, 20 );
        printf("Color value of dot @20.20 is %d.\n", color);
}
```

The program is compiled then linked. When the linker requests the library name, enter

+VIDCALLS. This includes the rdot() and wdot() modules at linking time. If this is not done, the linker will come back with unresolved externals error on the rdot() and wdot() functions.

```
; Microsoft C
; msc source;
; link source,,+vidcalls;
;
; TurboC
; tcc -c -ml -f- source.c
; tlink c0l source,source,source,cl vidcalls
```

ACCESSING MEMORY

The following program illustrates how to access memory. A **far pointer** called *scrn* is declared to point to the video RAM. This is actually accessed by use of the ES segment register. The program fills the video screen with the character A

```
#include <stdio.h> /* HACCESS1.C */
main()
{
        char far *scrn = (char far *) 0xB8000000;
        short int attribute = 164; /* Red on green blinking */
        int full_screen = 80 * 25 * 2, loop = 0;
        for( ; loop < full_screen; loop += 2 ) {
                scrn[ loop ] = 'A';
                scrn[ loop + 1 ] = attribute;
        }
        getchar();
}
```

The declaration of a far pointer specifies a 32 bit address. However the IBM-PC uses a 20 bit address bus. This 20 bit physical address is generated by adding the contents of a 16 bit offset to a SEGMENT register.

The Segment register contains a 16 bit value which is left shifted four times, then the 16 bit offset is added to this generating the 20 bit physical address.

```
Segment register = 0xB000 = 0xB0000
Offset = 0x0010 = 0x 0010
Actual 20 bit address = 0xB0010
```

When specifying the pointer using C, the full 32 bit combination of segment:offset is used, eg,

```
char far *memory_pointer = (char far *) 0xB0000000;
```

Lets consider some practical applications of this now. Located at segment 0x40, offset 0x4A is the number of columns used on the current video screen. The following program shows how to access this,

```
main()
{
        char far *video_parameters = (char far *) 0x00400000;
```

```
        int columns, offset = 0x4A;
        columns = video_parameters[offset];
        printf("The current column setting is %d\n", columns);
}
```

A practical program illustrating this follows. It directly accesses the video_parameter section of low memory using a far pointer, then prints out the actual mode using a message.

```
main() /* HACCESS2.C */
{
        static char *modes[] = { "40x25 BW", "40x25 CO", "80X25 BW",
        "80X25 CO", "320X200 CO", 320X200 BW",
        "640X200 BW", "80X25 Monitor" };
        char far *video_parameters = (char far *) 0x00400000;
        int crt_mode = 0x49, crt_columns = 0x4A;
        printf("The current video mode is %d\n",        modes[video_param
        printf("The current column width is %d\n", video_parameters[crt_c
}
```

An adaptation of this technique is the following two functions. **Poke** stores a byte value at the specified segment:offset, whilst **Peek** returns the byte value at the specified segment:offset pair.

```
void poke( unsigned int seg, unsigned int off, char value) {
        char far *memptr;
        memptr = (char far *) MK_FP( seg, off);
        *memptr = value;
}

char peek( unsigned int seg, unsigned int off ) {
        char far *memptr;
        memptr = (char far *) MK_FP( seg, off);
        return( *memptr );
}
```

The program **COLOR** illustrates the use of direct access to update the foreground color of a CGA card in text mode. It accepts the color on the command line, eg, COLOR RED will set the foreground color to RED.

```
#include <string.h> /* Color.c */
#include <dos.h>
#include <stdio.h>
#include <conio.h>
#define size 80*25*2

struct table {
        char *string;
        int value;
} colors[] = { {"BLACK" , 0, "BLUE" , 1 },
        {"GREEN" , 2, "CYAN" , 3 },
        {"RED" , 4, "MAGENTA" , 5 },
        {"BROWN" , 6, "LIGHT_GRAY" , 7 },
        {"DARK_GRAY" , 8, "LIGHT_BLUE" , 9 },
        {"LIGHT_GREEN" ,10, "LIGHT_CYAN" ,11 },
        {"LIGHT_RED" ,12, "LIGHT_MAGENTA" ,13 },
        {"YELLOW" ,14, "WHITE" ,15 } };

int getcolor( char *color) {
        char *temp;
```

```
        int loop, csize;
        temp = color;
        while( *temp )
                *temp++ = toupper( *temp );
        csize = sizeof(colors) / sizeof(colors[0]);
        for( loop = 0; loop < csize; loop++ )
                if( strcmp(color, colors[loop].string) == 0)
                        return(colors[loop].value;
        return( 16 );
}

void setcolor( int attr ) {
        int loop;
        char far *scrn = (char far *) 0xb8000000;
        for( loop = 1; loop < size; loop += 2 )
                scrn[loop] = attr;
}

main( int argc, char *argv[] ) {
        int ncolor;
        if( argc == 2 ) {
                ncolor = getcolor( argv[1] );
                if( ncolor == 16 )
                        printf("The color %s is not available.\n", argv[1
                else
                        setcolor( ncolor );
        }
}
```

POINTERS TO FUNCTIONS

Pointers to functions allow the creation of jump tables and dynamic routine selection. A pointer is assigned the start address of a function, thus, by typing the pointer name, program execution jumps to the routine pointed to. By using a single pointer, many different routines could be executed, simply by re-directing the pointer to point to another function. Thus, programs could use this to send information to a printer, console device, tape unit etc, simply by pointing the pointer associated with output to the appropiate output function!

The following program illustrates the use of pointers to functions, in creating a simple shell program which can be used to specify the screen mode on a CGA system.

```
#include <stdio.h> /* Funcptr.c */
#include <dos.h>
#define dim(x) (sizeof(x) / sizeof(x[0]) )
#define GETMODE 15
#define SETMODE 0
#define VIDCALL 0X10
#define SCREEN40 1
#define SCREEN80 3
#define SCREEN320 4
#define SCREEN640 6
#define VID_BIOS_CALL(x) int86( VIDCALL, &x, &x )
int cls(), scr40(), scr80(), scr320(), scr640(), help(), shellquit();
union REGS regs;
struct command_table {
        char *cmd_name;
        int (*cmd_ptr) ();
}cmds[]={"40",scr40,"80",scr80,"320",scr320,"640",scr640,"HELP",help,"CLS
```

```
cls() {
        regs.h.ah = GETMODE; VID_BIOS_CALL( regs );
        regs.h.ah = SETMODE; VID_BIOS_CALL( regs );
}

scr40() {
        regs.h.ah = SETMODE;
        regs.h.al = SCREEN40;
        VID_BIOS_CALL( regs );
}

scr80() {
        regs.h.ah = SETMODE;
        regs.h.al = SCREEN80;
        VID_BIOS_CALL( regs );
}

scr320() {
        regs.h.ah = SETMODE;
        regs.h.al = SCREEN320;
        VID_BIOS_CALL( regs );
}

scr640() {
        regs.h.ah = SETMODE;
        regs.h.al = SCREEN640;
        VID_BIOS_CALL( regs );
}

shellquit() {
        exit( 0 );
}

help() {
        cls();
        printf("The available commands are; \n");
        printf(" 40 Sets 40 column mode\n");
        printf(" 80 Sets 80 column mode\n");
        printf(" 320 Sets medium res graphics mode\n");
        printf(" 640 Sets high res graphics mode\n");
        printf(" CLS Clears the display screen\n");
        printf(" HELP These messages\n");
        printf(" EXIT Return to DOS\n");
}

get_command( char *buffer ) {
        printf("\nShell: ");
        gets( buffer );
        strupr( buffer );
}

execute_command( char *cmd_string ) {
        int i, j;
        for( i = 0; i < dim( cmds); i++ ) {
                j = strcmp( cmds[i].cmd_name, cmd_string );
                if( j == 0 ) {
                        (*cmds[i].cmd_ptr) ();
                        return 1;
                }
        }
        return 0;
}
```

```
main() {
        char input_buffer[81];
        while( 1 ) {
                get_command( input_buffer );
                if( execute_command( input_buffer ) == 0 )
                        help();
        }
}
```

INDEX Next